ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ И ВЕБ-ИНТЕРФЕЙСА

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		И.А. Лаврешин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков проектирования приложения интернета вещей и интерфейса для отображения состояния системы.

2 Задание и исходные данные

В рамках предложенной (или своей) темы спроектируйте систему, состоящую из 4-5 уникальных классов. Подумайте имеют ли классы общие методы и свойства, если таковые есть, выделите их в абстрактный класс. Обозначьте связь элементов системы. Подумайте какие данные в системе будет хранить база данных, обозначьте её отдельной сущностью, связями обозначьте какие данные и с каких вещей она должна хранить. Поскольку следующие лабораторные подразумевают написание кода на Python, в котором модификаторы доступа носят рекомендательный характер, ими можно пренебречь.

В рамках предложенной темы спроектируйте, как минимум два интерфейса. Два интерфейса могут быть нужны, например, для пользователя и администратора. На примере умного дома это может быть интерфейс ребёнка, который только отображает некоторые данные и позволяет управлять некоторыми из вещей и интерфейс родителя, которые может позволять более глубоко настраивать управление домом. Подумайте какие данные лучше скрыть от одного пользователя, какие лучше показать, как лучше их отобразить.

Вариант задания номер 9: Умное игровое поле для игры в баскетбол

3 Ход выполнения задания

В основе архитектуры лежит абстрактный класс `Thing`, который задаёт общий контракт для всей периферии: идентификатор, имя и способ получить текстовое состояние устройства через `getStatus()`. От него наследуются четыре класса. `ScoreSensor` отвечает за фиксацию попадания, `ZoneSensor` — за зону броска, чтобы отличать два и три очка, `EnvSensor` собирает параметры окружающей среды на площадке (температура и освещённость), а `Scoreboard` представляет табло с текущим счётом, временем и выводом на дисплей. Центральный класс `MainControlUnit` не является физическим устройством: он регистрирует список «вещей», опрашивает их, по данным датчика попадания и зоны броска пересчитывает счёт, командует обновлением табло и передаёт в `CourtDataStore` информацию для журнала событий и архива телеметрии. Таким образом, на одной схеме видно и иерархию устройств, и то, как управляющий блок связывает игровую логику с хранением данных.

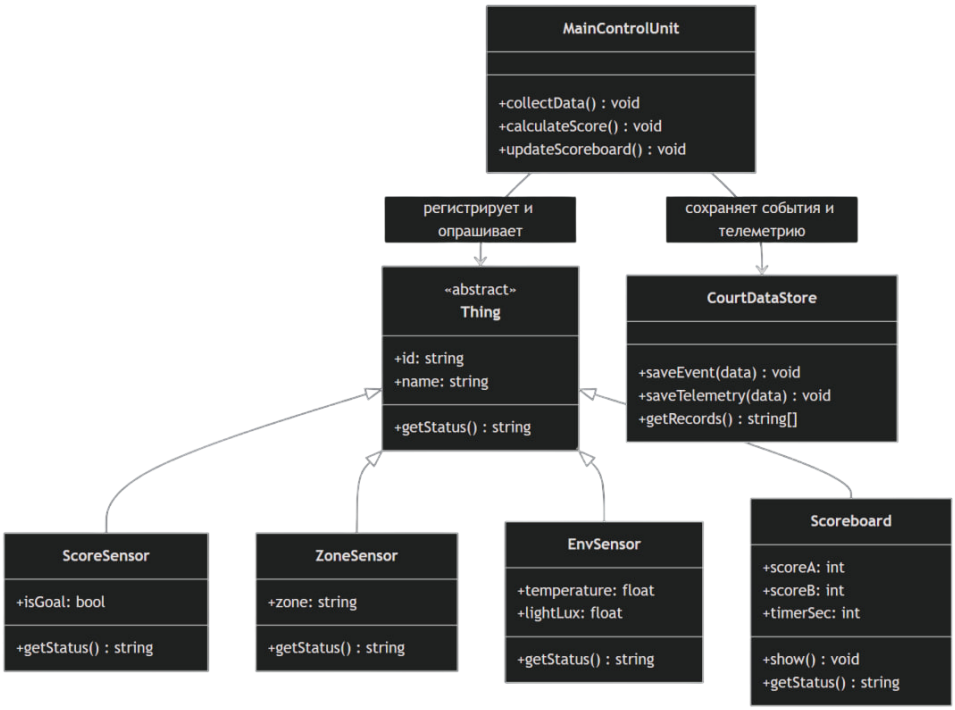

Рисунок 1 – Реализованная диаграмма классов

Для системы спроектированы два веб-интерфейса с разным уровнем доступа. Первый ориентирован на игрока и зрителя: на экране крупно показываются счёт команд, таймер текущего периода и короткая строка статуса матча (например, номер четверти или пауза), **без** кнопок управления и **без** служебных чисел с датчиков — чтобы информация читалась с расстояния и не отвлекала от игры. Второй интерфейс предназначен для администратора и судьи: здесь есть управление временем (старт, пауза, сброс периода), при необходимости ручная коррекция счёта, отдельный блок с текущими показаниями `EnvSensor` и журнал событий в хронологическом порядке (время, команда, начисленные очки), что позволяет контролировать работу автоматики и разбирать спорные моменты. Такое разделение соответствует требованию методички иметь не менее двух интерфейсов и отражает разные роли пользователей при одной и той же серверной логике.

Интерфейс игрока/зрителя: крупный счёт, таймер периода, краткий статус матча, только просмотр.

Интерфейс администратора/судьи: старт/пауза/сброс таймера, коррекция счёта при необходимости, телеметрия `EnvSensor`, журнал событий.

4 Вывод

В ходе выполнения лабораторной работы было осуществлено проектирование программной архитектуры системы баскетбольного табло с автоматическим учетом очков, строго соответствующее требованиям методического руководства. Разработанная структура демонстрирует глубинный подход к объектно-ориентированному проектированию, где основное внимание уделено разделению ответственности, расширяемости и моделированию различных пользовательских сценариев.